

ECON*6140 ECONOMETRICS I
Python Syntax Companion

Writing Assignment #1 yourself, one line at a time

Fall 2026 — works with the `.venv` in the `Metrics HW1` folder

What this is. A syntax manual, not a solutions manual. It shows you the Python you have to type — how to load the data, build a lag, run a regression, pull a number out of the output, plot a series, and form a linear combination of coefficients and its standard error. The econometric reasoning stays with you; the typing should not be what slows you down.

How to read it. Every grey block is a snippet that runs as written. Copy a block into a cell in `HW1.ipynb`, change the names, press **Shift+Enter**. Do not paste all of it at once — run one idea per cell so that when something breaks you know which line broke it.

Contents

1	The five objects you are actually working with	3
1.1	Attributes versus methods — the single most common beginner error	3
1.2	How to explore instead of memorise	3
2	Reading the spreadsheet	4
2.1	Turning the period column into a date index	4
3	Building the regression variables	4
3.1	What <code>shift</code> does to the top of the sample	5
3.2	Adding several columns at once	5
4	Choosing the estimation sample	5
5	Running a regression	6
5.1	The formula interface — use this one	6
5.2	The matrix interface — use it in question (5)	7
6	Getting numbers out of a fitted result	7
7	Plotting against time	8
8	Saving fitted values and residuals as data	9
8.1	Question (3): the two residual vectors	9
9	Linear combinations of coefficients: $\hat{\gamma} = w' \hat{\beta}$	9
9.1	A free check, and a second way to compute the same thing	10
10	The reparameterisation trick	10
11	Projection matrices in code	11
12	Errors you will actually hit	12

13 Small habits that will save you hours	13
14 Quick reference	14

1 The five objects you are actually working with

Almost every line of the assignment manipulates one of five things. Learn their names and the rest is vocabulary.

Object	What it is, and why you care
DataFrame	Your worksheet. Rows are observations, columns are variables. <code>df</code> in the notebook is one.
Series	A single column, or any single vector of numbers. <code>df['r']</code> is one; so is the residual vector of a regression.
Index	The row labels. Yours will be quarterly dates, which is what lets you write <code>loc['1950Q4':'1996Q4']</code> to slice a sample.
ndarray	A raw numpy array — numbers with no labels and no dates. The linear algebra in question (1) and the <code>w</code> vector in question (4) live here.
result object	What <code>.fit()</code> hands back (<code>res</code>). It is a bundle: parameters, standard errors, residuals, fitted values, covariance matrix.

1.1 Attributes versus methods — the single most common beginner error

An *attribute* is a stored fact, written with no parentheses. A *method* is an action, written with parentheses.

```
1 df.shape           # (188, 7)      attribute: parentheses would error
2 df.columns         # column names, as an Index
3 df.index           # the 188 date labels
4 df.dtypes          # what type each column is
5
6 df.head()          # method: the () is what calls it
7 df.head(3)         # an argument inside the () narrows it
8 df['r'].mean()      # method on a Series
```

If you ever see `TypeError: 'DataFrame' object is not callable`, you put parentheses on an attribute. If you see a bound-method printed at you (something like `<bound method ...>`), you left the parentheses off a method.

1.2 How to explore instead of memorise

You do not need to remember the whole library. You need to know how to ask it questions. In a notebook cell:

```
1 res.params         # what is in this object?
2 dir(res)           # every attribute and method it has
3 help(res.cov_params) # the documentation, in an output cell
4 res.cov_params?    # same thing, Jupyter shorthand
```

In a running cell, typing `res.` and pressing `Tab` lists what is available. If a name is half-typed, `Shift+Tab` shows its signature. This is faster than any cheat sheet, including this one.

2 Reading the spreadsheet

The data file is `tbrate.xlsx`, converted from the `tbrate.docx` on the course page. It has two sheets, `data` and `notes`; the working sheet holds 188 quarterly rows with columns `obs`, `year`, `quarter`, `period`, `r`, `y`, `pi`.

```
1 import numpy as np
2 import pandas as pd
3 import statsmodels.api as sm
4 import statsmodels.formula.api as smf
5 import matplotlib.pyplot as plt
6
7 pd.set_option('display.precision', 4)    # tidy output while you work
8
9 df = pd.read_excel('tbrate.xlsx', sheet_name='data')
10 df.head()
```

`sheet_name` takes the sheet's name as a string, or its position as a number (0 is the first sheet). `read_excel` needs the `openpyxl` engine, which is already in your `requirements.txt`.

2.1 Turning the period column into a date index

The `period` column arrives as text such as `'1950Q1'`. Convert it to a `PeriodIndex` and the frame gains a real time axis:

```
1 df.index = pd.PeriodIndex(df['period'], freq='Q')
2 df.index[:3]    # PeriodIndex(['1950Q1', '1950Q2', '1950Q3'], ...)
```

The `freq='Q'` is not optional decoration: it is what tells pandas the step between labels is one quarter, and therefore what `loc['1950Q4':'1996Q4']` means. Two consequences you will use constantly:

- **Slicing a sample by label.** `df.loc['1950Q4':'1996Q4']` keeps the rows from 1950Q4 to 1996Q4 inclusive. Both endpoints are included — unlike positional slicing, which excludes the last.
- **Plotting with dates on the x -axis.** Convert when you plot with `df.index.to_timestamp()`; a `PeriodIndex` does not always cooperate with matplotlib directly.

An equivalent tidier idiom, if you prefer the date column not to be duplicated as a variable:

```
1 df = df.set_index(pd.PeriodIndex(df['period'], freq='Q'))
```

3 Building the regression variables

The assignment writes the model in differences and lags. pandas does this with two vectorised operations — no loops, ever:

- `.diff()` — the first difference, $\Delta x_t = x_t - x_{t-1}$.
- `.shift(1)` — the one-period lag, x_{t-1} .

```

1 df['dr'] = df['r'].diff()           # Delta r_t = r_t - r_{t-1}
2 df['dy'] = df['y'].diff()           # Delta y_t = y_t - y_{t-1}
3 df['pi_1'] = df['pi'].shift(1)       # pi_{t-1}
4 df['dy_1'] = df['dy'].shift(1)       # Delta y_{t-1}
5 df['dr_1'] = df['dr'].shift(1)       # Delta r_{t-1}
6 df['dr_2'] = df['dr'].shift(2)       # Delta r_{t-2}

```

Three things worth internalising here.

Order matters. `dy_1` is defined from `dy`, so the `df['dy'] = ...` line must run before the `df['dy_1'] = ...` line. Writing them out of order gives you a column of `NaN` and no error message.

Chaining is allowed and equivalent. The series `dy_1` above is the same as `df['y'].diff()` with a `.shift(1)` applied afterwards: difference first, then lag. Read a chain left to right as English.

Lags are defined on the variable, not on the equation. Δr_{t-1} means “the series Δr , moved down one row”, which is `df['dr'].shift(1)`. It does *not* mean `df['r'].diff(1)` again.

3.1 What shift does to the top of the sample

`shift(1)` slides the series down one row and leaves a hole at the top, which pandas fills with a missing value, `NaN`:

```

1 df[['r', 'dr', 'pi_1', 'dy_1', 'dr_1', 'dr_2']].head(4)
2 #           r      dr  pi_1  dy_1  dr_1  dr_2
3 # 1950Q1 0.510  NaN   NaN   NaN   NaN   NaN
4 # 1950Q2 0.510 0.000 0.50   NaN   NaN   NaN
5 # 1950Q3 0.550 0.040 4.06   0.00 0.000  NaN
6 # 1950Q4 0.623 0.073 5.50   0.023 0.040  0.0

```

This is why the assignment’s sample starts at 1950:4 rather than 1950:1 — the first rows cannot carry a Δr_{t-2} , so they are not observations of the model. `NaN` propagates: any arithmetic touching a `NaN` returns `NaN`.

3.2 Adding several columns at once

`assign` returns a new frame, which is useful for a script but easy to get wrong in a notebook because it does not modify `df` in place:

```

1 df = df.assign(dr=df['r'].diff(), dy=df['y'].diff())

```

The reassignment `df = ...` is the part people forget. Plain square-bracket assignment, `df['dr'] = ...`, modifies `df` directly and is what you want most of the time.

4 Choosing the estimation sample

Two steps: cut the dates, then throw away the rows that the lags left incomplete.

```

1 sub = df.loc['1950Q4':'1996Q4'].copy()           # 1. the sample period
2 sub = sub.dropna(subset=['dr', 'pi_1', 'dy_1', 'dr_1', 'dr_2']) # 2. complete rows

```

```
3 sub.shape           # (185, 13) -- 185 usable observations
```

`dropna(subset=[...])` drops a row only if one of the *named* columns is missing; other columns may be NaN and the row survives. Called with no argument, `dropna()` drops rows with a missing value anywhere, which can silently delete data you meant to keep.

`.copy()` detaches `sub` from `df`. Without it, later lines like `sub['e'] = ...` raise a `SettingWithCopyWarning` because pandas cannot tell whether you meant to write into `df` as well.

A very useful check before every regression — the number of observations you think you have versus the number you actually have:

```
1 len(sub)                # 185
2 sub[['dr', 'pi_1']].isna().sum()  # missing values per column: 0 and 0
```

The single most common way to get two regressions that “should” match and do not is to let them run on different row counts, because a NaN hides in one of them. Keep an eye on `res.nobs`.

5 Running a regression

5.1 The formula interface — use this one

```
1 res = smf.ols('dr ~ pi_1 + dy_1 + dr_1 + dr_2', data=sub).fit()
2 res.summary()
```

The formula reads as “`dr` is explained by `pi_1`, `dy_1`, `dr_1`, `dr_2`”. Grammar you will need:

Formula	Meaning
<code>y ~ x1 + x2</code>	Regress <code>y</code> on <code>x1</code> , <code>x2</code> and an intercept. The <code>+</code> is <i>not</i> addition — it means “include as a regressor”.
<code>y ~ x1 - 1</code>	No intercept. Question (3) needs this: “regress Δr_t on a constant, Δy_{t-1} , ...” includes the constant, so you write nothing special; but suppressing one is <code>- 1</code> , not <code>+ 0</code> .
<code>np.log(x)</code>	Functions work inside a formula; <code>smf</code> looks them up in the data.
<code>I(x**2)</code>	Wrap arithmetic that is not a plain variable name in <code>I()</code> .
<code>y ~ x1 * x2</code>	Main effects <i>and</i> interaction.

Two things the formula interface does for you: it adds the constant automatically (so never add your own `const` column as well — that is a singular design matrix), and it drops incomplete rows and tells you it did.

Do not forget `.fit()`. `smf.ols(...)` alone is a model *specification*; `res` only exists after `.fit()`. Everything in the next section lives on the fitted result.

5.2 The matrix interface — use it in question (5)

When the regressors are not columns of a frame — when `X1`, `X2`, `P1y`, `M1y` are vectors you constructed — the array interface is the one to reach for:

```
1 X = sub[['pi_1', 'dy_1', 'dr_1', 'dr_2']]      # a DataFrame of regressors
2 X = sm.add_constant(X)                        # prepend the constant yourself
3 res = sm.OLS(sub['dr'], X).fit()
```

Note the differences from `smf`: `sm.OLS` takes the data as two arguments, not a formula; `sm.add_constant` is mandatory, because no formula is there to add the intercept; and the two inputs must line up row for row — same length, same order.

The estimator itself, if you want to see the algebra of question (1) in code:

```
1 Xv = np.asarray(X)                          # ndarray, shape (185, 5)
2 yv = sub['dr'].to_numpy()                    # ndarray, shape (185,)
3
4 b = np.linalg.inv(Xv.T @ Xv) @ Xv.T @ yv    # b = (X'X)^{-1} X'y
5 b
```

`@` is matrix multiplication; `*` is elementwise multiplication and will silently give you the wrong answer if you swap them. `Xv.T` is the transpose. The result agrees with `res.params.values` to machine precision. This is also your check on question (5): you can build any projection by hand and confirm the same numbers fall out.

6 Getting numbers out of a fitted result

`res` is a bundle of answers. These are the pieces the assignment asks for:

Expression	What it gives you
<code>res.params</code>	Coefficients, as a Series indexed by name: <code>res.params['pi_1']</code> .
<code>res.params.values</code>	The same numbers as a plain array, in formula order. Use this for matrix work.
<code>res.bse</code>	Standard errors, indexed the same way.
<code>res.tvalues</code> , <code>res.pvalues</code>	t statistics and two-sided p -values.
<code>res.cov_params()</code>	The $k \times k$ covariance matrix $\widehat{\text{Var}}(b)$. A method — it needs the parentheses.
<code>res.conf_int()</code>	95% confidence intervals, one row per coefficient.
<code>res.resid</code>	Residuals $\hat{u}_t$, a Series with the sample's date index.
<code>res.fittedvalues</code>	Fitted values $\hat{y}_t$, likewise.
<code>res.nobs</code>	Number of observations actually used.
<code>res.rsquared</code>	R^2 ; <code>res.rsquared_adj</code> is the adjusted version.
<code>res.df_resid</code>	Residual degrees of freedom $n - k$.

Expression	What it gives you
<code>res.model.exog</code>	The design matrix as used, with the constant in column 1.
<code>res.summary()</code>	The printed table.

Reading `res.summary()`: the top block is the model, the middle block is the coefficient table (coefficient, standard error, t , p , interval — one row per regressor), and the bottom block is R^2 , F , log-likelihood, AIC and BIC. Scan the middle block for the numbers the questions ask for, and the bottom block for No. `Observations`.

`res.resid` and `res.fittedvalues` are ordinary Series — you can take their mean, their correlation, or regress one on the other. That is the whole of question (2).

7 Plotting against time

```

1 fig, ax = plt.subplots(figsize=(9, 3.5))
2 ax.plot(sub.index.to_timestamp(), sub['resid'], lw=1.0, color='C0')
3 ax.axhline(0, color='0.6', lw=0.8)           # a zero reference line
4 ax.set_title('OLS residuals')
5 ax.set_xlabel('year')
6 ax.set_ylabel('residual')
7 fig.tight_layout()
8 plt.show()

```

The pattern — make a figure, draw on the axes, label, show — is always the same. To get residuals and fitted values on two stacked panels that share one time axis (which is what “plot against time” wants):

```

1 fig, axes = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
2 t = sub.index.to_timestamp()
3
4 axes[0].plot(t, sub['fit'], lw=1.0)
5 axes[1].plot(t, sub['resid'], lw=1.0)
6 axes[0].set_ylabel('fitted'); axes[1].set_ylabel('residual')
7 axes[1].set_xlabel('year')
8 fig.tight_layout()
9 plt.show()

```

`axes` is a list here (two panels, so two axes); index it with `axes[0]` and `axes[1]`. `sharex=True` makes them share the time axis so they line up visually — important when the point of the plot is to see whether the residual spike coincides with a fitted spike.

To keep a figure for the write-up:

```

1 fig.savefig('q2_residuals.png', dpi=200, bbox_inches='tight')

```

8 Saving fitted values and residuals as data

Question (2) and question (3) both need a residual vector to become a *column* so you can regress it on something. Assign it into the frame you estimated from:

```
1 sub = sub.copy()                # once, before adding columns
2 sub['resid'] = res.resid         # aligned by date index
3 sub['fit'] = res.fittedvalues
```

This works because `res.resid` carries the same date index as `sub`, and pandas aligns on the index rather than on position. That alignment is a feature — and a trap: if you assign a residual from a regression run on a *different* sample, the shorter vector will not line up and you get NaNs rather than an error. When in doubt, check `sub['resid'].isna().sum()`.

Then the two auxiliary regressions of question (2) are just regressions:

```
1 smf.ols('resid ~ fit', data=sub).fit().params
2 smf.ols('fit ~ resid', data=sub).fit().params
```

Read three numbers from each: the slope on the right-hand variable, its p -value, and `res.rsquared`. Then ask whether the second regression can reconstruct the first, and what $R^2 = 0$ means when you have regressed one series on another.

8.1 Question (3): the two residual vectors

The pattern is identical: estimate an auxiliary regression, harvest `.resid`, store it, and run the last regression on the stored columns. The only way question (3) goes wrong is sample mismatch, so build both auxiliary regressions from the same `sub` and the same `dropna`:

```
1 ra = smf.ols('dr ~ dy_1 + dr_1 + dr_2', data=sub).fit()
2 sub['e'] = ra.resid
3
4 rb = smf.ols('pi_1 ~ dy_1 + dr_1 + dr_2', data=sub).fit()
5 sub['epi'] = rb.resid
6
7 rc = smf.ols('e ~ epi', data=sub).fit()
8 rc.params['epi']      # compare this with the coefficient from question (2)
9 rc.resid              # compare these with the residuals from question (2)
```

Printing the two numbers side by side is the fastest way to see the relationship:

```
1 print(rc.params['epi'], res.params['pi_1'])
2 print(np.allclose(rc.resid, res.resid))    # True or False: are they identical?
```

9 Linear combinations of coefficients: $\hat{\gamma} = w' \hat{\beta}$

Question (4) asks for $\hat{\gamma} = \sum_{i=2}^6 \hat{\beta}_i$ and its standard error. You know the rule $\text{Var}(\hat{\gamma}) = w' \text{Var}(\hat{\beta}) w$; the only new skill is writing w in code.

```
1 b = res.params.values          # array of length k, in formula order
```

```

2 V = np.asarray(res.cov_params()) # k x k covariance matrix
3
4 w = np.zeros(len(b))           # start with all zeros...
5 w[1:6] = 1                     # ...and put a 1 on the coefficients you want
6
7 gamma_hat = w @ b               # the point estimate
8 se_gamma = np.sqrt(w @ V @ w)  # its standard error
9 print(gamma_hat, se_gamma)

```

The three lines that matter:

- `np.zeros(len(b))` makes a vector of the right length, all zeros. A w that is the wrong length is the most common error here, so build it from `len(b)` rather than typing `np.zeros(6)`.
- `w[1:6] = 1` sets the ones. Python counts from zero, so slot 0 is the intercept and slots 1...5 are the first five slopes. Read off `res.params.index` to see exactly which name sits in which slot: `print(list(res.params.index))`.
- `np.sqrt(w @ V @ w)` is the scalar whose square root is the standard error. $w'Vw$ is a quadratic form and comes out as a one-element array; `np.sqrt` turns it into the number you want.

If `res.params` were built from a plain numpy design matrix without names, the same recipe works — only the indexing is by position, so be certain of your column order.

9.1 A free check, and a second way to compute the same thing

statsmodels can impose the linear restriction for you, which is exactly the $w'\hat{\beta}$ calculation:

```

1 tt = res.t_test('pi_1 + dy_1 = 0') # or any linear combination
2 float(np.ravel(tt.effect)[0])       # the combination
3 float(np.ravel(tt.sd)[0])           # its standard error
4 float(np.ravel(tt.pvalue)[0])       # p-value for the restriction

```

The restriction is written as a string in the same grammar as the formula: coefficient names, +, -, and = 0. The `np.ravel(...)[0]` wrapper is needed because `effect`, `sd` and `pvalue` come back as one-element arrays.

Use it to *verify* your w -route answer, not to replace it: if the two disagree, your w has a one in the wrong slot. That check is worth doing once, and then you can trust the w method for the case where the restriction is not a simple sum.

10 The reparameterisation trick

The second method question (4) asks for is a transformation that makes $\hat{\gamma}$ a coefficient of the regression, so that its standard error comes straight off the printout. The algebra is yours to write; here is the syntax, on a small example that has the same shape.

Suppose the model is $y = \beta_1 + \beta_2 x + \beta_3 z_1$ (three regressors, and you want $\gamma = \beta_2 + \beta_3$). Define a new variable $u_1 = z_1 - x$. Substituting $z_1 = x + u_1$ into the model gives $\beta_2 x + \beta_3 z_1 = (\beta_2 + \beta_3)x + \beta_3 u_1$: the coefficient on x is now γ itself.

```

1 d = d.copy()
2 d['u1'] = d['z1'] - d['x']          # build the new regressor
3
4 res_orig = smf.ols('y ~ x + z1', data=d).fit()
5 res_new = smf.ols('y ~ x + u1', data=d).fit()
6
7 res_orig.params['x'] + res_orig.params['z1']  # gamma the hard way
8 res_new.params['x']                        # gamma, read off the output
9 res_new.bse['x']                          # its standard error, likewise

```

Three properties of this transformation, all of which you can confirm on the example above and which are the reason the trick works:

- The two regressions have *identical fitted values* — the model is reparameterised, not changed. Check with `np.allclose(res_orig.fittedvalues, res_new.fittedvalues)`.
- $\hat{\gamma}$ from `res_new.params['x']` equals $\hat{\beta}_2 + \hat{\beta}_3$ from the original, exactly.
- Its standard error from `res_new.bse['x']` equals $\sqrt{w'Vw}$ from the previous section, exactly.

To apply this to question (4), extend the same construction mechanically: one new variable per coefficient you are absorbing into γ , each equal to the original variable minus the variable whose coefficient becomes γ . Build the new columns in the frame, then write a formula that names them. Then answer the natural question — why must the fitted values not change, when the regressors did?

11 Projection matrices in code

Question (5) hands you P_1 , $M_1 = I - P_1$, P_X and $M_X = I - P_X$, and asks which of ten auxiliary regressions reproduce $\hat{\beta}_2$ and which reproduce the residuals. Two ways to build them.

The literal way — form the matrices exactly as written:

```

1 n = len(d)
2 X1 = np.column_stack([np.ones(n), d['x'].to_numpy()]) # [1, x]
3 P1 = X1 @ np.linalg.inv(X1.T @ X1) @ X1.T           # n x n
4 M1 = np.eye(n) - P1                                 # n x n

```

`np.column_stack` glues vectors side by side into a matrix; `np.eye(n)` is the $n \times n$ identity. Note that P_1 and M_1 are 185×185 here — the arithmetic is fine, but it is wasteful.

The practical way — a projection is a fitted value. $P_1 y$ is the vector of fitted values from regressing y on X_1 , and $M_1 y$ is that regression's residual. So:

```

1 r1 = smf.ols('y ~ x', data=d).fit()
2 P1y = r1.fittedvalues.to_numpy() # = P1 @ y
3 M1y = r1.resid.to_numpy()        # = M1 @ y = y - P1y
4
5 rall = smf.ols('y ~ x + z1 + z2', data=d).fit()
6 PXy = rall.fittedvalues.to_numpy() # = PX @ y
7 MXy = rall.resid.to_numpy()        # = MX @ y

```

This is the version to use for (5). Every left-hand side of (a)–(j) is one of y , P_1y , M_1y , P_Xy — all four are now ordinary vectors in your frame. Every right-hand side is a variable, or P_1X_2 , or M_1X_2 , MX_X2 — and a projected *regressor* such as M_1X_2 is likewise just a residual series, one column of X_2 at a time:

```
1 r2 = smf.ols('z1 ~ x', data=d).fit()
2 d['M1z1'] = r2.resid.to_numpy()      # the first column of M1 @ X2
```

Then each auxiliary regression is one line, and you compare results in a small dictionary rather than by eye:

```
1 out = {}
2 out['original'] = rall.params['z1']
3 out['(a)'] = smf.ols('y ~ z1', data=d).fit().params['z1']
4 out['(f)'] = smf.ols('M1y ~ z1', data=d).fit().params['z1']
5 out['(g)'] = smf.ols('M1y ~ M1z1', data=d).fit().params['z1']
6 print(out)
```

Watch for the cases where a right-hand side has no constant in it. The formula interface adds an intercept unless you write `- 1`, and whether an intercept belongs is precisely part of what the question is probing — so decide deliberately, and if a case is ambiguous under the standard formula interface, that is a signal to use `sm.OLS` with a hand-built design matrix, where you control the columns exactly.

12 Errors you will actually hit

Message	Cause	Fix
KeyError: 'dr'	The column does not exist — the line that creates it was not run, or was run out of order.	Run the cells from the top. Check <code>df.columns</code> .
exog contains inf or nans	A regressor has a missing or infinite value inside the estimation sample.	<code>dropna(subset=[...])</code> before the regression. Check for <code>np.log(0)</code> , which returns <code>-inf</code> .
LinAlgError: Singular matrix	The regressors are perfectly collinear — most often the constant added twice, or an n -column X_1 that already spans the whole design.	Never <code>add_constant</code> a matrix whose first column is already ones. Check <code>np.linalg.matrix_rank(Xv)</code> against the column count.
Two coefficients that should match do not	The two regressions used different samples.	Print <code>res.nobs</code> for both. Rebuild both from the same <code>sub</code> .
SettingWithCopyWarning	You assigned a new column into a frame that is a slice of another frame.	<code>sub = sub.copy()</code> once, up front.

Message	Cause	Fix
ValueError: cannot insert period, already exists	<code>reset_index()</code> tried to create a column with a name that is already there.	Drop the duplicate first, or rename the index before resetting.
TypeError: 'DataFrame' object is not callable	Parentheses on an attribute: <code>df.shape()</code> .	Attributes take no arguments.
ValueError: operands could not be broadcast	Arrays of different lengths — usually a NaN-trimmed <code>res.resid</code> versus the full frame.	Print <code>len()</code> of both, and align by index rather than by position.
R^2 of exactly 1.0	You regressed a series on itself, — for example using <code>fit</code> as the regressor when the left-hand side is <code>fit</code> .	Check the variable names on both sides of the $\sim$.
Nothing prints after <code>.fit()</code>	<code>.fit()</code> returns the result; it does not display it in a script.	In a script, <code>print(res.summary());</code> in a notebook, put it on the last line of the cell.

13 Small habits that will save you hours

- **One idea per cell.** A cell that builds one variable and prints `.head()` tells you immediately when the variable is wrong. A cell that builds six columns tells you nothing.
- **Print the shape, not just the values.** `df.shape` after every construction step catches an off-by-one in the sample before it becomes an off-by-one in a standard error.
- **Name variables after the assignment's symbols.** `dr`, `pi_1`, `dy_1`, `dr_2`, `e`, `epi`, `dc`, `gamma`. When you compare your output to the question sheet you will not have to translate.
- **Verify a linear algebra result with a library call.** Build $\hat{\gamma}$ with `w` and with `t_test`; build `b` with `sm.OLS` and with `np.linalg.inv`. Two independent routes agreeing is a proof; one route looking reasonable is not.
- **Check the one thing that must not change.** The reparameterisation must leave fitted values untouched, and the FWL auxiliary regressions must leave certain coefficients untouched. Test that invariant with `np.allclose` — it is a much sharper test than eyeballing two tables, and it tells you *which* of the ten cases in (5) behave that way.
- **Send numbers to LaTeX, not to your typing hand.** When the assignment asks you to report a table, `res.summary().as_latex()` and `sub[['dr']].head().to_latex()` produce paste-ready

output for your write-up.

14 Quick reference

Call	What it does	Example
<code>pd.read_excel</code>	Read a spreadsheet sheet into a frame	<code>pd.read_excel('tbrate.xlsx', sheet_name='data')</code>
<code>pd.PeriodIndex</code>	Build a quarterly date index	<code>pd.PeriodIndex(df['period'], freq='Q')</code>
<code>DataFrame.loc</code>	Select rows and columns by label	<code>df.loc['1950Q4':'1996Q4', 'r']</code>
<code>Series.diff</code>	First difference, Δx_t	<code>df['r'].diff()</code>
<code>Series.shift</code>	Lag by s periods, x_{t-s}	<code>df['pi'].shift(1)</code>
<code>DataFrame.dropna</code>	Drop rows with missing values	<code>df.dropna(subset=['dr', 'pi_1'])</code>
<code>DataFrame.copy</code>	Detach a slice from its parent	<code>sub = df.loc[...].copy()</code>
<code>DataFrame.assign</code>	Add columns, returning a new frame	<code>df = df.assign(dy=df['y'].diff())</code>
<code>smf.ols</code>	OLS from a formula	<code>smf.ols('dr ~ pi_1', data=sub).fit()</code>
<code>sm.OLS</code>	OLS from an array design matrix	<code>sm.OLS(y, X).fit()</code>
<code>sm.add_constant</code>	Prepend a column of ones	<code>sm.add_constant(X)</code>
<code>result.params</code>	Coefficients	<code>res.params['pi_1']</code>
<code>result.bse</code>	Standard errors	<code>res.bse['dr_1']</code>
<code>result.cov_params</code>	$k \times k$ covariance matrix	<code>V = np.asarray(res.cov_params())</code>
<code>result.resid</code>	Residuals	<code>sub['e'] = res.resid</code>
<code>result.fittedvalues</code>	Fitted values	<code>sub['fit'] = res.fittedvalues</code>
<code>result.t_test</code>	Test a linear restriction	<code>res.t_test('b1 + b2 = 0')</code>
<code>np.zeros</code>	Vector of zeros, for w	<code>w = np.zeros(len(b))</code>
<code>np.sqrt</code>	Square root, for the std. error	<code>np.sqrt(w @ V @ w)</code>
<code>np.allclose</code>	Compare two vectors numerically	<code>np.allclose(a, b)</code>
<code>np.linalg.inv</code>	Matrix inverse	<code>np.linalg.inv(X.T @ X)</code>
<code>np.column_stack</code>	Glue vectors into a matrix	<code>np.column_stack([v1, v2])</code>
<code>plt.subplots</code>	Figure plus axes	<code>fig, ax = plt.subplots(figsize=(9,3.5))</code>
<code>Axes.plot</code>	Draw a line	<code>ax.plot(t, y, lw=1.0)</code>
<code>Axes.axhline</code>	Horizontal reference line	<code>ax.axhline(0, color='0.6')</code>

Call	What it does	Example
<code>Figure.savefig</code>	Write the figure to a file	<code>fig.savefig('q2.png', dpi=200)</code>

One last thing. The econometrics of questions (1) and (5) is algebra, and code will not do it for you. What code does is let you *check* the algebra: derive the partitioned-inverse expressions on paper, then build M_1 and M_2 in numpy and confirm the formula. That is the habit worth taking away from this assignment — the computer as a check on the proof, not a substitute for it.